

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчет
по лабораторной работе № 8
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Обход графа в ширину»

Выполнили
студенты группы 24ВВВ2:

Макаров Я.С.

Бегичев Д.Д.

Родионов К.Е.

Приняли

Юрова О.В.

Деев М.В.

Пенза, 2025

Цель работы

Научиться производить обход в графа в ширину.

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности

для неориентированного графа G. Выведите матрицу на экран.

2. Для сгенерированного графа осуществите процедуру обхода в ширину,

реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс `queue` из стандартной библиотеки C++.

3.* Реализуйте процедуру обхода в ширину для графа, представленного списками смежности.

Задание 2*

1. Для матричной формы представления графов реализуйте алгоритм обхода

в ширину с использованием очереди, построенной на основе структуры данных «список», самостоятельно созданной в лабораторной работе № 3

2. Оцените время работы двух реализаций алгоритмов обхода в ширину (использующего стандартный класс `queue` и использующего очередь, реализованную самостоятельно) для графов разных порядков.

Описание метода решения

Метод решения заданий заключается в следующем: сначала генерируется случайный неориентированный граф заданного размера и строятся две его формы представления — матрица смежности и списки смежности. Затем реализуется обход графа в ширину (BFS), который начинается с выбранной стартовой вершины. Для обхода BFS используется очередь: в первом задании — стандартная `std::queue` из C++, во втором — самодельная очередь на основе односвязного списка с сортировкой элементов по приоритету. Основная идея BFS — из очереди берется вершина для обработки, затем перебираются все её соседи в графе; каждый непосещенный сосед помечается как посещенный и добавляется в очередь для последующего обхода. Такой подход обеспечивает последовательный обход графа "слоями" — сначала вершины, непосредственно смежные со стартовой, затем их соседи и так далее. Во втором задании дополнительно проводится замер времени работы обхода с обеими реализациями очереди, что демонстрирует различие в производительности. В итоге реализуется эффективный и структурированный обход графа, позволяющий исследовать все достижимые вершины из начальной, с акцентом на понимание работы очереди и внутреннего представления графа.

Псевдокод

1-ое задание

алг обход_графа_и_операции

дано

- число вершин V
- матрица смежности $matrica$ размера $V \times V$
- списки смежности $adjLists$
- стартовая вершина $start$

надо

- выполнить обход в ширину (BFS) для матрицы и списков
- вывести результаты обхода

нач

ввод $V, start$

создать матрицу смежности и списки смежности (генерировать или вручную)

вывести матрицу смежности и списки смежности

// BFS — матрица

процедура $BFS_Matrix(start)$

нач

создать массив $visited$ длиной V , заполнить $False$

создать очередь q

$visited[start] := True$

поместить $start$ в q

обойти пока q не пуст

извлечь $current$ из q

вывести $current + 1$

для i от 0 до $V-1$

если $matrica[current][i] = 1$ и $not\ visited[i]$ тогда

visited[i] := True

поместить i в q

конец если

конец цикл

кон

// BFS — списки смежности

процедура BFS_List(start)

нач

создать массив visited длиной V, заполнить False

создать очередь q

visited[start] := True

поместить start в q

обойти пока q не пуст

извлечь current из q

вывести current + 1

для каждого соседа в списке adjLists[current]

если не посещён тогда

visited[сосед] := True

поместить соседа в q

конец если

конец цикл

кон

// Вывод результата

процедура printBFSResult(result, method)

нач

вывести "Результат обхода (" + method + "): "

для каждого элемента в result

вывести элемент + 1

если не последний => вывести " -> "

кон

вывести перевод строки

кон

// В main:

вывести результаты BFS с матрицы и списков, вызывая соответствующие процедуры

кон

2-ое задание

алг обход_графа_матрица

дано

- число вершин V
- матрица смежности $matrica$ размером $V \times V$

надо

- выполнить обход в ширину (BFS) двумя способами: с `std::queue` и с собственной приоритетной очередью
- измерить время выполнения и вывести результаты

нач

V , $matrica$, результат, очередь, очередь_приоритетная, `visited`, `start`,
время_начала, время_конца

// BFS с `std::queue`

процедура `BFS_матрица_std_queue(start)`

нач

отметить все вершины как не посещённые

создать очередь q

`visited[start] := true`

начать таймер

```
поместить start в q
пока q не пуст
current := извлечь из q
добавить current в результат
для i от 0 до V-1
если есть ребро (matrica[current][i] = 1) и не посещена i тогда
visited[i] := true
поместить i в q
конец если
конец для
конец пока
остановить таймер
вывести время = время_конца - время_начала в микросекундах
кон
```

```
// BFS с собственной приоритетной очередью (стеком)
процедура BFS_матрица_приоритет_очередь(start)
нач
все вершины не посещены
создать собственную очереди (Приоритетный стек)
visited[start] := true
поместить start в очередь_приоритетную с приоритетом 0
начать таймер
пока очередь_приоритетная не пуста
current := извлечь вершину из очереди_приоритетной
добавить current в результат
для i от 0 до V-1
если есть ребро (matrica[current][i] = 1) и не посещена i тогда
visited[i] := true
поместить i в очередь_приоритетную с приоритетом 0
конец если
```

конец для

конец пока

остановить таймер

вывести время = время_конца - время_начала и результаты

конец

// Основной блок

нач

ввести V, стартовую вершину

вызвать BFS_матрица_std_queue(старт) и вывести время

вызвать BFS_матрица_приоритет_очередь(старт) и вывести время

конец

Листинг

1-ое задание

```
#include <iostream>
#include <vector>
#include <queue>
#include <random>
#include <iomanip>
#include <string>
using namespace std;
struct SMEZHLIST {
    int dest;
    SMEZHLIST* next;
    SMEZHLIST(int d): dest(d), next(nullptr) {}
};
struct AdjList {
    SMEZHLIST* head;
};
class Graph {
private:
    int V;
    vector<vector<int>> adjMatrix;
    vector<AdjList> adjLists;

public:
    Graph(int vertices): V(vertices) {
        adjMatrix.resize(V, vector<int>(V, 0));
        adjLists.resize(V);
        for (int i = 0; i < V; ++i)
            adjLists[i].head = nullptr;
    }

    ~Graph() {
        for (int i = 0; i < V; ++i) {
            SMEZHLIST* curr = adjLists[i].head;
            while (curr) {
```

```

SMEZHLIST* temp = curr;
curr = curr->next;
delete temp;
}
}
}

void genrandgraph() {
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<> dis(0, 1);

    for (int i = 0; i < V; ++i) {
        for (int j = i + 1; j < V; ++j) {
            int edge = dis(gen);
            adjMatrix[i][j] = edge;
            adjMatrix[j][i] = edge;
        }
    }
    spissmezhhotmatr();
}

void spissmezhhotmatr() {
    for (int i = 0; i < V; ++i) {
        SMEZHLIST* curr = adjLists[i].head;
        while (curr) {
            SMEZHLIST* temp = curr;
            curr = curr->next;
            delete temp;
        }
        adjLists[i].head = nullptr;
    }

    for (int i = 0; i < V; ++i) {
        for (int j = 0; j < V; ++j) {
            if (adjMatrix[i][j] == 1)
                rebrovspis(i, j);
        }
    }
}

```

```
}  
}
```

```
void rebrovspis(int src, int dest) {  
    SMEZHLIST* newNode = new SMEZHLIST(dest);  
    newNode->next = adjLists[src].head;  
    adjLists[src].head = newNode;  
}
```

```
void pringsmezhmatr() {  
    cout << "Матрица смежности:\n ";  
    for (int i = 0; i < V; ++i)  
        cout << setw(3) << i;  
    cout << "\n";  
    for (int i = 0; i < V; ++i) {  
        cout << setw(2) << i;  
        for (int j = 0; j < V; ++j) {  
            cout << setw(3) << adjMatrix[i][j];  
        }  
        cout << "\n";  
    }  
    cout << "\n";  
}
```

```
void pringsmezhspis() {  
    cout << "Списки смежности:\n";  
    for (int i = 0; i < V; ++i) {  
        cout << i << ": ";  
        SMEZHLIST* curr = adjLists[i].head;  
        while (curr) {  
            cout << curr->dest;  
            if (curr->next) cout << " -> ";  
            curr = curr->next;  
        }  
        cout << "\n";  
    }  
}
```

```

cout << "\n";
}

vector<int> BFS_Matrix_StdQueue(int start) {
vector<int> result;
vector<bool> visited(V, false);
queue<int> q;

visited[start] = true;
q.push(start);

while (!q.empty()) {
int current = q.front();
q.pop();
result.push_back(current);

for (int i = 0; i < V; ++i) {
if (adjMatrix[current][i] == 1 && !visited[i]) {
visited[i] = true;
q.push(i);
}
}
}

return result;
}

vector<int> BFS_List_StdQueue(int start) {
vector<int> result;
vector<bool> visited(V, false);
queue<int> q;

visited[start] = true;
q.push(start);

while (!q.empty()) {

```

```
int current = q.front();
q.pop();
result.push_back(current);
```

```
SMEZHLIST* neighbor = adjLists[current].head;
while (neighbor) {
    if (!visited[neighbor->dest]) {
        visited[neighbor->dest] = true;
        q.push(neighbor->dest);
    }
    neighbor = neighbor->next;
}
}
```

```
return result;
}
```

```
static void printBFSResult(const vector<int>& result, const
string& method) {
    cout << "Результат обхода (" << method << "): ";
    for (size_t i = 0; i < result.size(); ++i) {
        cout << result[i];
        if (i != result.size() - 1) cout << " -> ";
    }
    cout << "\n\n";
}
};
```

```
int safeInputInt(const string& prompt, int minVal, int maxVal) {
    int value;
    while (true) {
        cout << prompt;
        cin >> value;
        if (cin.fail() || value < minVal || value > maxVal) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

```

cout << "Ошибка: введите число в диапазоне " << minVal << " - "
<< maxVal << "!\n";
} else {
cin.ignore(numeric_limits<streamsize>::max(), '\n');
return value;
}
}
}

int main() {
setlocale(LC_ALL, "Russian");
int V = safeInputInt("Введите количество вершин графа: ", 1,
10000);
int start = safeInputInt("Введите стартовую вершину (0-" +
to_string(V-1) + "): ", 0, V-1);

Graph g(V);
g.genrandgraph();

g.pringsmezhmatr();
g.pringsmezhspis();

cout << "BFS для матрицы смежности с std::queue:\n";
auto res1 = g.BFS_Matrix_StdQueue(start);
Graph::printBFSResult(res1, "матрица + std::queue");

cout << "BFS для списков смежности с std::queue:\n";
auto res2 = g.BFS_List_StdQueue(start);
Graph::printBFSResult(res2, "списки + std::queue");

return 0;
}

```

2-ое задание

```

#include <iostream>
#include <vector>

```

```

#include <string>
#include <queue>
#include <chrono>
#include <random>
#include <iomanip>
using namespace std;
using namespace std::chrono;
struct Node {
    string name;
    int priority;
    Node* next;
    Node(const string& n, int p) : name(n), priority(p),
next(nullptr) {}
};
class PriorityQueue {
private:
    Node* head;
    Node* tail;

public:
    PriorityQueue() : head(nullptr), tail(nullptr) {}
    ~PriorityQueue() {
        clear();
    }
    void push(const string& name, int priority) {
        Node* newNode = new Node(name, priority);
        if (!head) {
            head = tail = newNode;
        }
        else {
            if (priority < head->priority) {
                newNode->next = head;
            }
        }
    }
};

```

```

        head = newNode;
    }
    else {
        Node* current = head;
        while (current->next && current->next->priority
<= priority)

            current = current->next;
        newNode->next = current->next;
        current->next = newNode;
        if (!newNode->next)
            tail = newNode;
    }
}

```

```

void pop() {
    if (!head) return;
    Node* temp = head;
    head = head->next;
    if (!head) tail = nullptr;
    delete temp;
}

```

```

Node* front() const {
    return head;
}

```

```

bool empty() const {
    return head == nullptr;
}

```

```

void clear() {

```



```

        while (head) pop();
    }
};

class Graph {
private:
    int V;
    vector<vector<int>> adjMatrix;

public:
    Graph(int vertices) : V(vertices) {
        adjMatrix.resize(V, vector<int>(V, 0));
    }

    void generateRandomGraph() {
        random_device rd;
        mt19937 gen(rd());
        uniform_int_distribution<> dis(0, 1);
        for (int i = 0; i < V; ++i) {
            for (int j = i + 1; j < V; ++j) {
                int edge = dis(gen);
                adjMatrix[i][j] = edge;
                adjMatrix[j][i] = edge;
            }
        }
    }

    void printAdjMatrix() {
        cout << "Матрица смежности:\n ";
        for (int i = 0; i < V; ++i)
            cout << setw(3) << i;
        cout << "\n";
        for (int i = 0; i < V; ++i) {

```

```

        cout << setw(2) << i;
        for (int j = 0; j < V; ++j) {
            cout << setw(3) << adjMatrix[i][j];
        }
        cout << "\n";
    }
    cout << "\n";
}

vector<int> BFS_Matrix_StdQueue(int start) {
    vector<int> result;
    vector<bool> visited(V, false);
    std::queue<int> q;
    auto start_time = high_resolution_clock::now();
    visited[start] = true;
    q.push(start);
    while (!q.empty()) {
        int current = q.front();
        q.pop();
        result.push_back(current);

        for (int i = 0; i < V; ++i) {
            if (adjMatrix[current][i] == 1 && !visited[i]) {
                visited[i] = true;
                q.push(i);
            }
        }
    }

    auto end_time = high_resolution_clock::now();
    auto duration = duration_cast<microseconds>(end_time -
start_time);

    cout << "Время обхода (матрица + std::queue): " <<
duration.count() << " мкс" << endl;
}

```

```

        return result;
    }

    vector<int> BFS_Matrix_CustomQueue(int start) {
        vector<int> result;
        vector<bool> visited(V, false);
        PriorityQueue pq;
        auto start_time = high_resolution_clock::now();
        visited[start] = true;
        pq.push(to_string(start), 0);
        while (!pq.empty()) {
            Node* currentNode = pq.front();
            int currentVertex = stoi(currentNode->name);
            result.push_back(currentVertex);
            pq.pop();

            for (int i = 0; i < V; ++i) {
                if (adjMatrix[currentVertex][i] == 1 &&
!visited[i]) {
                    visited[i] = true;
                    pq.push(to_string(i), 0);
                }
            }
        }

        auto end_time = high_resolution_clock::now();
        auto duration = duration_cast<microseconds>(end_time -
start_time);

        cout << "Время обхода (матрица + приоритетная очередь):
" << duration.count() << " мкс" << endl;

        return result;
    }
};

int main() {

```

```
    setlocale(LC_ALL, "Russian");  
    int V;  
    cout << "Введите количество вершин графа: ";  
    cin >> V;  
    Graph g(V);  
    g.generateRandomGraph();  
    g.printAdjMatrix();  
    int start;  
    cout << "Введите стартовую вершину (0-" << V - 1 << "): ";  
    cin >> start;  
    g.BFS_Matrix_StdQueue(start);  
    g.BFS_Matrix_CustomQueue(start);  
    return 0;  
}
```

Результат работы программы

1-ое:

```
Консоль отладки Microsoft Vi X + v
Введите количество вершин графа: 5
Введите стартовую вершину (0-4): 0
Матрица смежности:
  0 1 2 3 4
0 0 0 0 1 1
1 0 0 1 0 1
2 0 1 0 1 0
3 1 0 1 0 1
4 1 1 0 1 0

Списки смежности:
0: 4 -> 3
1: 4 -> 2
2: 3 -> 1
3: 4 -> 2 -> 0
4: 3 -> 1 -> 0

BFS для матрицы смежности с std::queue:
Результат обхода (матрица + std::queue): 0 -> 3 -> 4 -> 2 -> 1

BFS для списков смежности с std::queue:
Результат обхода (списки + std::queue): 0 -> 4 -> 3 -> 1 -> 2
```

2-ое:

```
Консоль отладки Microsoft Vi X + v
Введите количество вершин графа: 5
Матрица смежности:
  0 1 2 3 4
0 0 1 0 1 1
1 1 0 1 0 1
2 0 1 0 0 1
3 1 0 0 0 0
4 1 1 1 0 0

Введите стартовую вершину (0-4): 0
Время обхода (матрица + std::queue): 19 мкс
Время обхода (матрица + приоритетная очередь): 25 мкс
```

Вывод

В ходе выполнения лабораторной работы были успешно реализованы и протестированы различные подходы к обходу графов в ширину с использованием различных структур данных. Разработанная программа включает четыре основные реализации алгоритма: обход с матрицей смежности и стандартной очередью, обход со списками смежности и стандартной очередью, обход с матрицей смежности и приоритетной очередью, а также обход со списками смежности и приоритетной очередью.